

STATIC APPLICATION SECURITY TESTING · PENTEST REPORT

Informe SAST · Módulo Admin

Scope: src/app/admin/** · src/app/api/admin/**

Framework: Next.js 15 (App Router) · TypeScript · Prisma · shadcn/ui

ESTÁNDAR	HALLAZGOS TOTALES	FECHA DE EMISIÓN	ENGINE
OWASP Top 10 2021	9 vulnerabilidades	2026-08-12T00:56:49.695Z	puppeteer-core · Manual SAST

RESUMEN DE SEVERIDAD



ÍNDICE DE HALLAZGOS

#1	IDOR + Mass Assignment en Actualización de Roles (Escalación Vertical)	Crítico
#2	Escalación Horizontal: Admin Dashboard lee datos GLOBALES sin scoping (Cross-Tenant Leak)	Crítico
#3	Mass Assignment vía Spread Operator de permissions sin allow-list estricta	Alto
#4	Validación de roles inconsistente (Role Collision) entre rutas gemelas	Alto
#5	Guardia de autorización sin filtro status: APPROVED	Alto
#6	invitationToken de 32 bytes retornado en texto plano por JSON	Medio
#7	members/:id/access expone error.message crudo de Prisma al cliente	Medio
#8	Páginas admin "use client" sin proteger server-side ante usuarios no autorizados	Bajo
#9	Rate limit ausente en mutaciones críticas (invitaciones, cambio de status)	Bajo

OWASP A01:2021 · BROKEN ACCESS CONTROL

Crítico

#1 IDOR + Mass Assignment en Actualización de Roles (Escalación Vertical)

Archivo y Línea exacta:

src/app/api/admin/users/[id]/route.ts [L51-L62, L100-L109]

⚠ Nivel de Riesgo:

Crítico

🔍 Descripción del Riesgo:

La ruta PATCH /api/admin/users/:id valida que el solicitante sea owner o ADMIN de la organización, pero no valida si el roleId enviado corresponde a un CustomRole existente dentro de la MISMA organización. Cualquier roleId de otra organización pasa la comprobación y su customRoleId se persiste sin pertenencia. Además, strings arbitrarios que no coinciden con [ADMIN, AUDITOR, VIEWER] caen por defecto en VIEWER con customRoleId arbitrario, permitiendo Privilege Escalation por desvío a roles de tenants vecinos.

✳ Ejemplo de Exploit:

```
PATCH /api/admin/users/2c0c...9ab5
{
  "roleId": "custom-role-robado-de-OTRA-org",
  "companyIds": []
}
```

Si el CustomRole robado tiene orgRoles: true o providerBusinessRules: true, el atacante hereda permisos elevados si

✅ Código Corregido:

```
const isSystemRole = ['ADMIN', 'AUDITOR', 'VIEWER'].includes(roleId)
let systemRole: 'ADMIN' | 'AUDITOR' | 'VIEWER' = 'VIEWER'
let customRoleId: string | null = null

if (isSystemRole) {
  systemRole = roleId as 'ADMIN' | 'AUDITOR' | 'VIEWER'
} else {
  const validCustomRole = await prisma.customRole.findUnique({
    where: { id: roleId },
    select: { id: true, organizationId: true }
  })
  if (!validCustomRole || validCustomRole.organizationId !== adminMembership.organizationId) {
    return NextResponse.json({ error: 'Rol personalizado inválido' }, { status: 400 })
  }
  customRoleId = validCustomRole.id
}
```

OWASP A01:2021 · BROKEN ACCESS CONTROL

Crítico

#2 Escalación Horizontal: Admin Dashboard lee datos GLOBALES sin scoping (Cross-Tenant Leak)

Archivo y Línea exacta:

src/app/api/admin/dashboard/route.ts [L38-L45, L48-L78]

⚠ Nivel de Riesgo:

Crítico

🔍 Descripción del Riesgo:

El handler valida Permission.ADMIN_DASHBOARD en el systemRole del usuario, pero las consultas a Prisma (count, findMany, groupBy) NO filtran por organizationId. Esto expone al admin de la Organización A el conteo total de usuarios del sistema, empresas de tenants vecinos, auditLogs con PII (emails, RFCs), agregados de industrias y estados globales. Violación severa de aislamiento multi-tenant.

✳ Ejemplo de Exploit:

1. Iniciar sesión con cuenta normal SystemRole=ADMIN de Org-A.
 2. GET /api/admin/dashboard.
 3. Response incluye:
 - totalUsers (de TODAS las orgs + platform owners)
 - totalCompanies / recentCompanies (con createdBy userIds)
 - recentAuditLogs con userEmail, companyName, description y RFCs
 - topIndustries / topStates agregados de todo el SaaS.
- PII expuesto sin necesidad de estar autorizado en esos tenants.

✅ Código Corregido:

```
// Obtener membresía autorizada con su organizationId
const member = await prisma.member.findFirst({
  where: { userId: user.id, status: 'APPROVED' },
  select: { organizationId: true }
})
if (!member) return NextResponse.json({ error: 'Sin membresía' }, { status: 403 })
const orgId = member.organizationId

// Scopear TODAS las consultas
prisma.company.count({ where: { organizationId: orgId } }),
prisma.company.findMany({ where: { organizationId: orgId }, ... }),
prisma.auditLog.findMany({
  where: {
    timestamp: { gte: ... },
    OR: [
      { organizationId: orgId },
      { company: { organizationId: orgId } }
    ]
  }
})
```

OWASP A03:2021 · INJECTION

Alto

#3 Mass Assignment vía Spread Operator de permissions sin allow-list estricta

📄 Archivo y Línea exacta:

src/app/api/admin/roles/route.ts, src/app/api/admin/roles/[id]/route.ts [L192-L217 (POST), L24-L42 (PUT)]

⚠️ Nivel de Riesgo:

Alto

🔍 Descripción del Riesgo:

La creación/edición de CustomRole recibe body.permissions y lo inyecta directamente con ...permissions sin allow-list ni validación de schema. Si el modelo tiene campos que no debieran escribirse por API (id, organizationId, createdAt, flags internos), el spread permite sobrescribirlos u overpostear campos arbitrarios que coincidan con el schema de Prisma. El parser no rechaza llaves extra.

✳️ Ejemplo de Exploit:

```
POST /api/admin/roles
{
  "name": "Hacked Role",
  "permissions": {
    "organizationId": "uuid-ORG-AJENA",
    "canManageOrg": true,
    "createdAt": "1970-01-01T00:00:00.000Z"
  },
  "granularPermissions": { "orgRoles": true }
}
```

Dependiendo de laxitud de Prisma, se pueden colgar roles de otras organizaciones o manipular metadata no editable.

✅ Código Corregido:

```
import { z } from 'zod'

const PERMISSION_KEYS = [
  'canViewEmission', 'canViewReception', 'canViewPayroll',
  'canViewSatPortal', 'canViewMassDownloads', 'canManageOrg'
] as const

const createRoleSchema = z.object({
  name: z.string().min(1).max(100),
  description: z.string().max(500).optional(),
  granularPermissions: z.record(z.string(), z.boolean()).default({}),
  permissions: z.object({
    canViewEmission: z.boolean().default(false),
    canViewReception: z.boolean().default(false),
    canViewPayroll: z.boolean().default(false),
    canViewSatPortal: z.boolean().default(false),
    canViewMassDownloads: z.boolean().default(false),
    canManageOrg: z.boolean().default(false),
  }).strict() // Rechaza llaves desconocidas
})
```

```
  })

  const body = createRoleSchema.parse(await req.json())
  const newRole = await prisma.customRole.create({
    data: {
      organizationId: member.organizationId,
      name: body.name,
      description: body.description,
      ...body.permissions,
      granularPermissions: body.granularPermissions
    }
  })
})
```

OWASP A04:2021 · INSECURE DESIGN

Alto

#4 Validación de roles inconsistente (Role Collision) entre rutas gemelas

 Archivo y Línea exacta:`src/app/api/admin/members/[id]/route.ts, src/app/api/admin/members/[id]/access/route.ts [L48-L56, L95-L103]` Nivel de Riesgo:

Alto

 Descripción del Riesgo:

Existen 4 rutas que asignan roles (`users/:id`, `members/:id`, `members/:id/access` y `bulk-invite`). Sólo `bulk-invite` verifica que el `CustomRole` exista en la organización. Las otras 3 no centralizan validación, produciendo `Role Collision`: un mismo `roleId` se acepta por una ruta y es rechazado por otra. Rompe auditabilidad y crea surface de ataque desigual según qué endpoint use el atacante.

 Ejemplo de Exploit:

1. Intentar asignar un `CustomRole` de org ajena vía `bulk-invite` → rechazado.
2. Usar el mismo ID vía `PATCH /api/admin/members/:id` → aceptado.

El layout protegido no sirve si existe un endpoint "rápido" desprotegido con la misma funcionalidad.

 Código Corregido:

```
// src/lib/admin-roles.ts (helper centralizado)
export async function resolveRoleForOrg(
  roleId: string,
  organizationId: string
): Promise<{ systemRole: 'ADMIN'|'AUDITOR'|'VIEWER'; customRoleId: string | null }> {
  if (['ADMIN','AUDITOR','VIEWER'].includes(roleId)) {
    return { systemRole: roleId as any, customRoleId: null }
  }
  const cr = await prisma.customRole.findUnique({
    where: { id: roleId },
    select: { id: true, organizationId: true }
  })
  if (!cr || cr.organizationId !== organizationId) {
    throw new Error('Rol inválido para la organización')
  }
  return { systemRole: 'VIEWER', customRoleId: cr.id }
}

// Invocar en las 4 rutas (users, members, members/access, bulk-invite)
```

OWASP A07:2021 · IDENTIFICATION & AUTHENTICATION FAILURES

Alto

#5

Guardia de autorización sin filtro status: APPROVED

 Archivo y Línea exacta:

src/app/api/admin/roles/route.ts, src/app/api/admin/roles/[id]/route.ts [L184-L190, L59-L65]

 Nivel de Riesgo:**Alto** Descripción del Riesgo:

En roles/route.ts y roles/[id]/route.ts, la guardia findFirst(where: {userId}) no filtra status == "APPROVED". Si un admin es dado de baja (status INACTIVE/PENDING/ONBOARDING) pero su member.role sigue siendo ADMIN, las credenciales siguen pasando la guardia. Las otras rutas admin (users, invite, sat-69b/sync) sí filtran APPROVED, creando bypass selectivo.

 Ejemplo de Exploit:

1. Admin legítimo suspendido: status = INACTIVE, role = ADMIN.
2. Cookie de sesión / refresh token sigue vigente.
3. GET /api/admin/roles → PASA.
4. PUT /api/admin/roles/<id> → PUEDE editar roles y escalar privilegios mientras no se limpie el rol.

 Código Corregido:

```
const member = await prisma.member.findFirst({
  where: {
    userId: session.user.id,
    status: 'APPROVED' // Filtro OBLIGATORIO
  },
  include: { organization: true }
})
if (!member || member.role !== "ADMIN") {
  return NextResponse.json({ error: "Acceso denegado" }, { status: 403 })
}
```

#6 invitationToken de 32 bytes retornado en texto plano por JSON

📄 Archivo y Línea exacta:

src/app/api/admin/users/invite/route.ts [L70-L73]

⚠️ Nivel de Riesgo:

Medio

🔍 Descripción del Riesgo:

El endpoint POST /api/admin/users/invite retorna invitationToken en el body del response. El comentario "Returned ONLY once so the admin can copy it immediately" es falso: los reverse proxies (nginx, Cloudflare), APMs (Datadog, Sentry) y herramientas SIEM loguean bodies HTTP automáticamente. Además, el historial de DevTools del navegador lo cachea indefinidamente. Token se transmite por HTTP en dev sin seguridad de capa de transporte.

🌟 Ejemplo de Exploit:

Admin compartiendo pantalla en Zoom/Meet → tercero ve token en Network tab.
MITM en LAN insegura sin HSTS STS → captura body del response.
Operador de infraestructura con acceso a logs de balanceador puede extraer tokens de invitación y crear cuentas en

✅ Código Corregido:

```
return NextResponse.json({
  success: true,
  message: 'Usuario invitado exitosamente',
  existingUser: invitation.existingUser,
  // invitationToken: ELIMINADO. Se transmite UNICAMENTE por correo.
})

// Si se requiere copiado manual para admins con problemas de email:
// generar JWT firmado de 5 minutos de vida con payload hasheado
// y obligar a un flujo de confirmación 2FA o doble-click antes de emitirlo.
```


OWASP A02:2021 · CRYPTOGRAPHIC FAILURES / INFORMATION DISCLOSURE

Medio

#7 members/:id/access expone error.message crudo de Prisma al cliente

 Archivo y Línea exacta:`src/app/api/admin/members/[id]/access/route.ts [L140-L144]` Nivel de Riesgo:

Medio

 Descripción del Riesgo:

El handler PATCH hace `const message = error instanceof Error ? error.message : "..."` y devuelve `NextResponse.json({ error: message }, 500)`. Prisma emite errores como "Unique constraint failed on the fields: (memberId, companyId)", "Foreign key constraint failed on the field: companyAccess_memberId_fkey" o "column ... does not exist". El atacante mapea nombres de tablas, columnas y restricciones, reduciendo drásticamente el esfuerzo de fingerprint y orientando IDOR/SQLi

 Ejemplo de Exploit:

```
PATCH /api/admin/members/<id>/access
{ "companyId": "...", "role": "BAD_PAYLOAD_TO_FINGERPRINT" }
```

Response 500:

```
{"error":"Unique constraint failed on the fields: (memberId_companyId): The required connection Company on the Comp
```

Nombre de tablas y columnas extraídos sin esfuerzo.

 Código Corregido:

```
} catch (error) {
  console.error('[admin/members/access] Error interno:', error) // SOLO logs internos
  return NextResponse.json(
    { error: 'Error interno del servidor' },
    { status: 500 }
  )
}
```

OWASP A05:2021 · SECURITY MISCONFIGURATION

Bajo

#8 Páginas admin "use client" sin proteger server-side ante usuarios no autorizados

📄 Archivo y Línea exacta:

src/app/admin/users/page.tsx, roles/page.tsx, profiles/page.tsx, users-bulk/page.tsx [L1-L33, L1-L70, L1-L46, L1-L15]

⚠️ Nivel de Riesgo:

Bajo

🔍 Descripción del Riesgo:

Sólo admin/dashboard/page.tsx usa <ProtectedRoute> + <PermissionRequired>. Las otras 4 páginas son puras "use client" sin wrapper ni redirect server-side. Usuario sin permiso que conozca URL /admin/users carga shell completo de UI, botones, nombres de módulos y endpoints subyacentes. Expone superficie de ataque y metadata del sistema sin necesidad de 403 temprano.

✳️ Ejemplo de Exploit:

1. Usuario con role USER simple navega a /admin/roles (ruta conocida por leak de fuente o enum común).
2. Recibe UI completa del gestor de roles (Nombres de permisos ~60 switches organizados por módulo).
3. Fingerprint de features y enumeración de granularPermissions de otras orgs a través del layout visual.

✅ Código Corregido:

```
// Envolver cada página al igual que admin/dashboard
import { PermissionRequired } from '@components/auth/PermissionRequired'
import { ProtectedRoute } from '@components/auth/ProtectedRoute'
import { Permission } from '@lib/permissions'

return (
  <ProtectedRoute>
    <PermissionRequired permission={Permission.ADMIN_USERS}>
      { /* ... page JSX original ... */ }
    </PermissionRequired>
  </ProtectedRoute>
)
```

OWASP A05:2021 · SECURITY MISCONFIGURATION

Bajo

#9 Rate limit ausente en mutaciones críticas (invitaciones, cambio de status)

Archivo y Línea exacta:

src/app/api/admin/users/invite/[id]/route.ts, src/app/api/admin/users/[id]/status/route.ts [DELETE y PATCH handlers completos]

⚠ Nivel de Riesgo:

Bajo

🔍 Descripción del Riesgo:

DELETE /api/admin/users/invite/:id y PATCH /api/admin/users/:id/status no tienen rate limit por usuario autenticado. Aunque requieren autorización, un admin comprometido o XSS en el panel podrían iterar IDs visibles en listados (fuera del rango UUID inviable) para activar/inactivar usuarios en bucle sin backoff, causando denegación de servicio operativa.

✳ Ejemplo de Exploit:

1. Atacante roba sesión de admin y obtiene listado de userIds/memberIds por GET /api/admin/users.
2. Script en loop: 1000 llamadas por minuto al PATCH status=INACTIVE.
3. Ningún rate limiter frena: usuarios se inactivan en masa, denegando acceso a la organización completa.

✅ Código Corregido:

```
// src/lib/rate-limit.ts (importar y aplicar en cada handler)
import { rateLimitByUserId } from '@lib/rate-limit'

await rateLimitByUserId({
  userId: session.user.id,
  key: 'admin-status-patch',
  limit: 30,
  windowMs: 60_000 // 30 cambios por minuto por admin
})
```